Volcano for the Agentic AI Era:

Unified Scheduling Across Training, Inference, and Agents

Ricardo Aravena · Rainie Li

KubeCon + CloudNativeCon North America 2026
Salt Lake City · November 9–12

Speakers

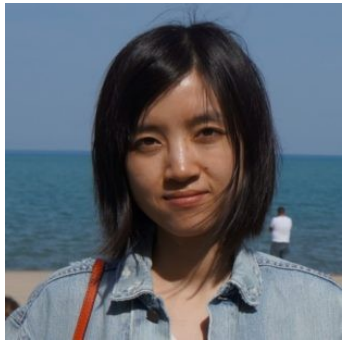


Ricardo Aravena

Cloud Native & AI Infrastructure

Open-source infrastructure
Kubernetes and AI/ML platforms

serverbooter.com



Rainie Li

Pinterest

Data processing infrastructure
Apache YuniKorn PMC member

Apache YuniKorn · rainieli

01

Unified scheduling

01 Unified scheduling

02 Multi-level gang & network topology

03 Ecosystem

04 Benchmark framework & progress

Different paths. Shared infrastructure.

How do schedulers coordinate node access?

Different Workloads, Shared Infrastructure

Training, inference, and agent applications put different demands on the same cluster.

Training & RL

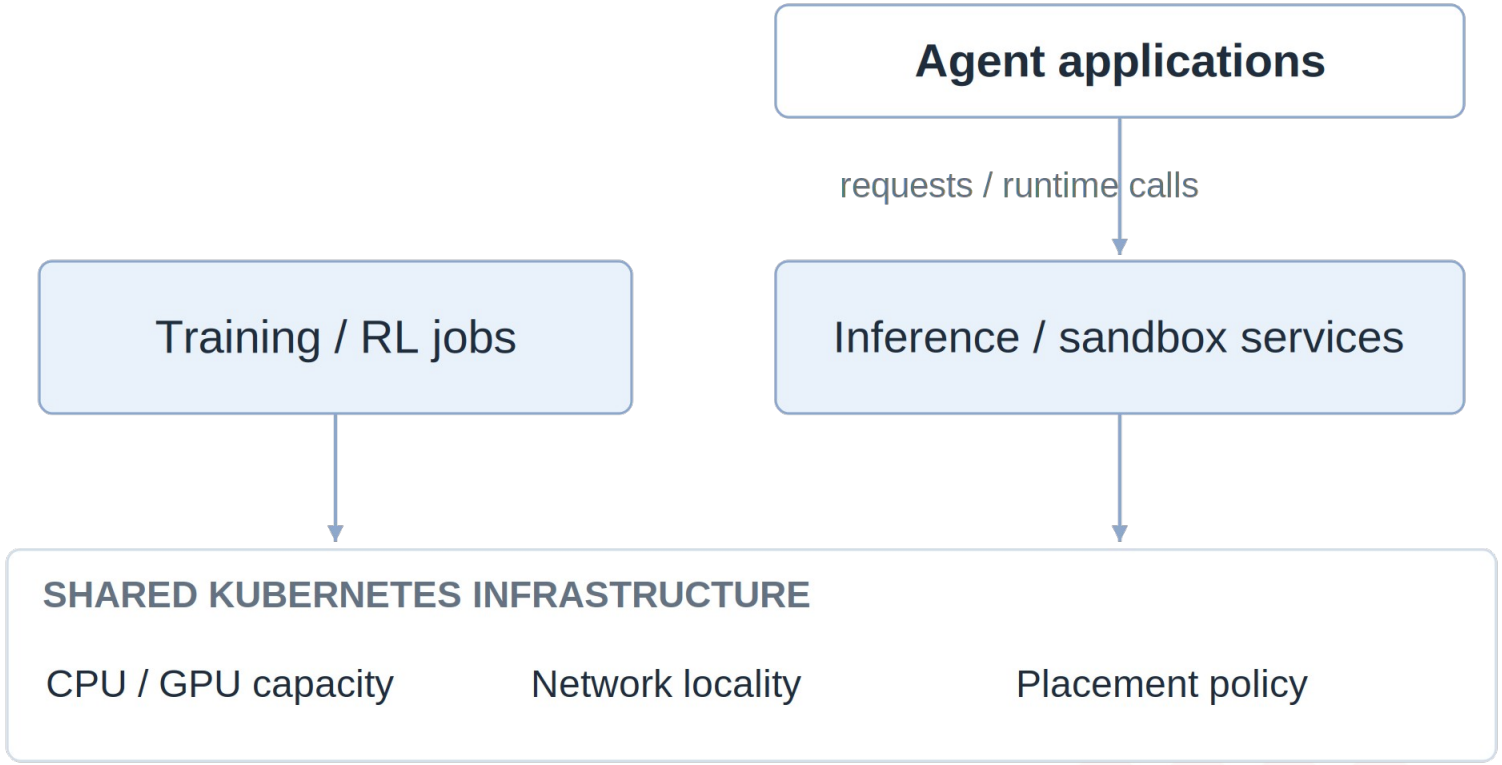
Coordinated compute

Inference

Serving roles and replicas

Agent applications

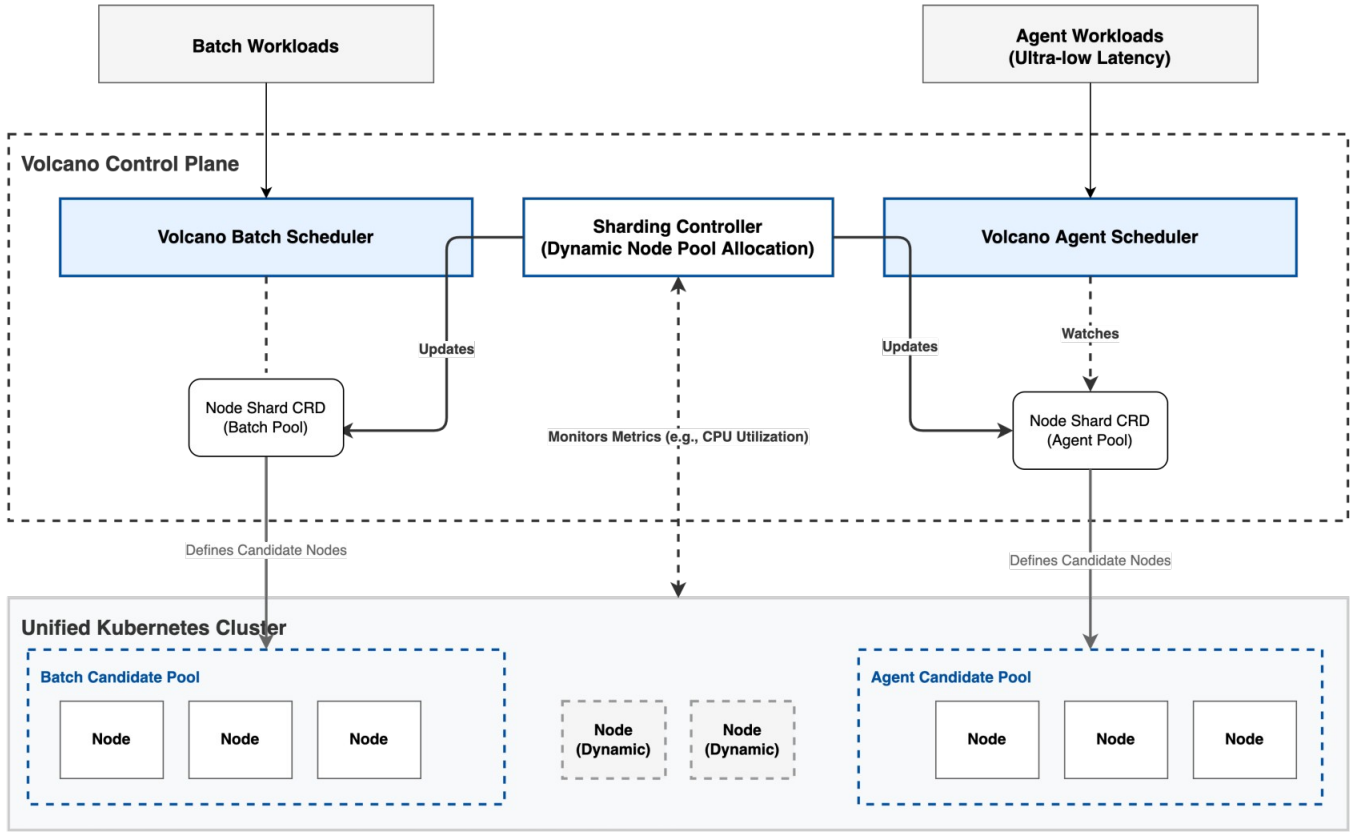
Bursty, latency-sensitive work



The scheduling unit depends on the runtime and workload model.

A Coordinated Scheduling Framework

Specialized scheduling paths coordinate through NodeShard.



01 Workload paths

Batch-oriented scheduling and a Pod-oriented fast path.

02 Local coordination

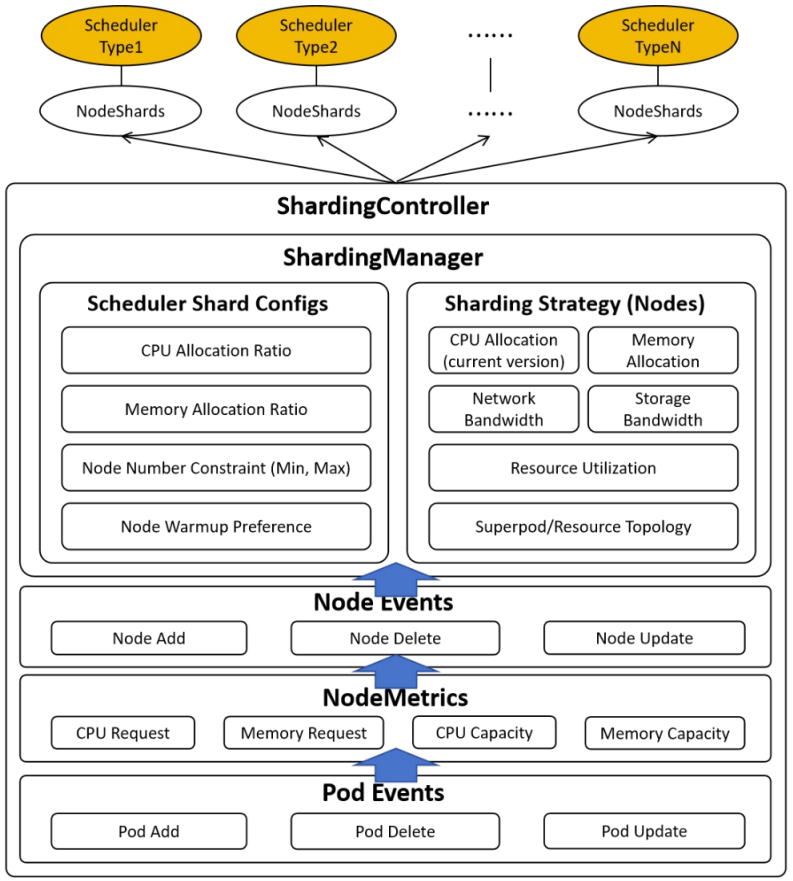
Parallel workers use Binder conflict checks.

03 Shared node access

NodeShard coordinates candidate-node pools.

NodeShard: Desired Nodes and Current Use

Changing a shard coordinates future placement; it does not migrate running Pods.



NodeShard · excerpt

```
kind: NodeShard
metadata:
  name: volcano
spec:
  nodesDesired: [node1, node2, node3]
status:
  nodesInUse: [node0, node1, node2]
  nodesToRemove: [node0]
  nodesToAdd: [node3]
```

Desired membership ≠ immediate use

Coordinators reconcile the handoff at scheduling boundaries.

02

Multi-level gang & network topology

From available nodes to runnable workloads.

What must run together, and where should it fit?

01 Unified scheduling

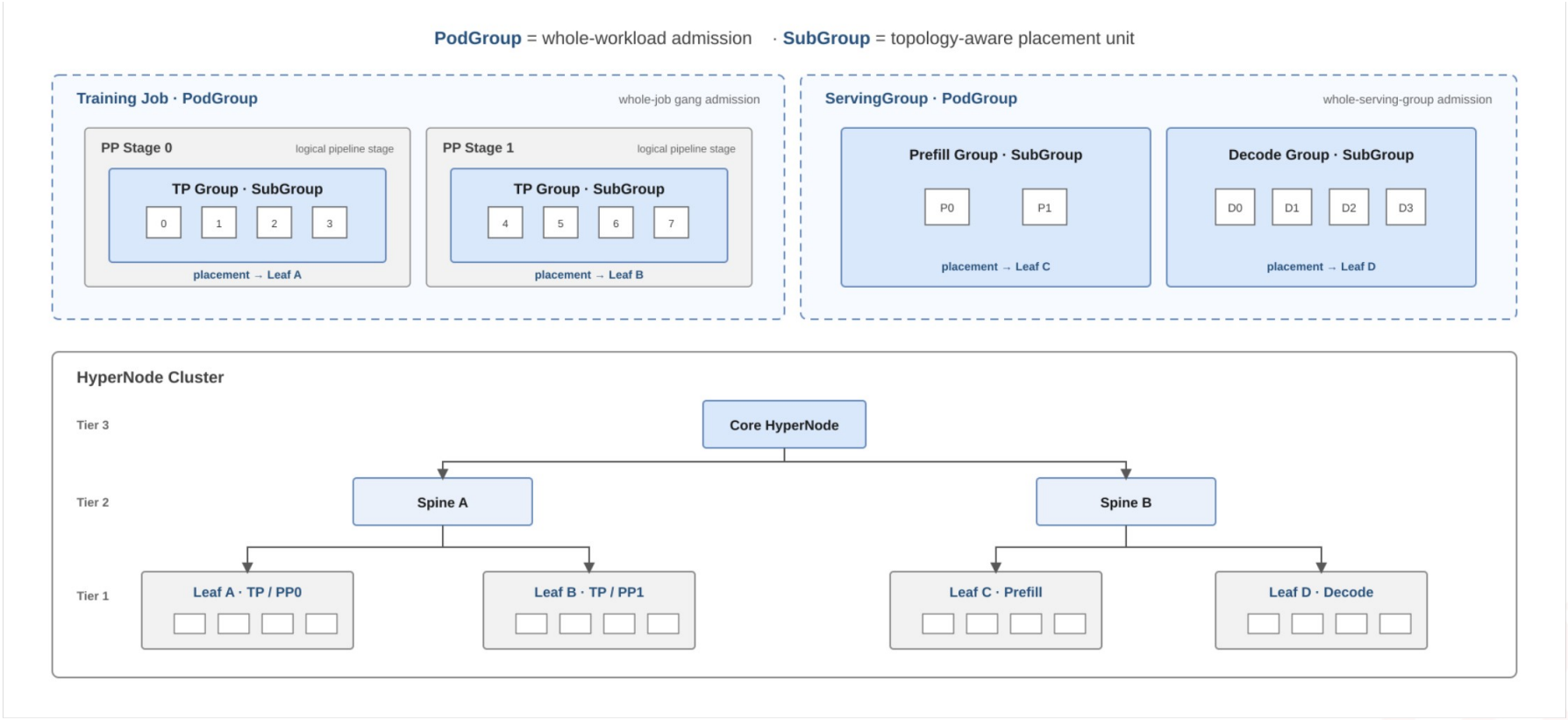
02 Multi-level gang & network topology

03 Ecosystem

04 Benchmark framework & progress

PodGroup and SubGroup: Preserve Workload Structure

Gang requirements apply to the workload and to its complete subgroups.

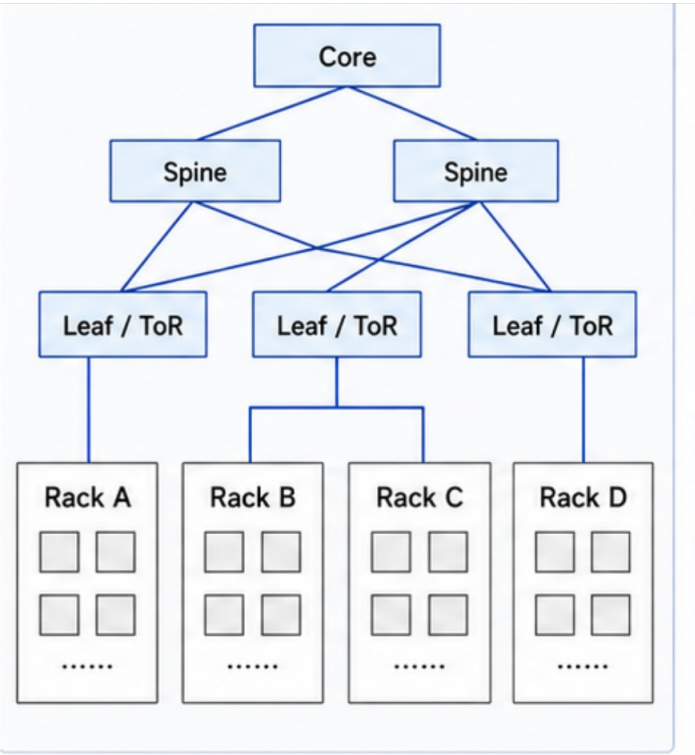


A SubGroup policy selects Pods; replica identity can split them into multiple SubGroups.

HyperNode: From Physical Fabric to Scheduling Domains

A parent HyperNode groups child HyperNodes; a leaf groups Kubernetes Nodes.

PHYSICAL FABRIC



Parent · HyperNode.spec

```
tier: 2
tierName: spine
members:
- type: HyperNode
  selector:
    exactMatch:
      name: rack-a
```

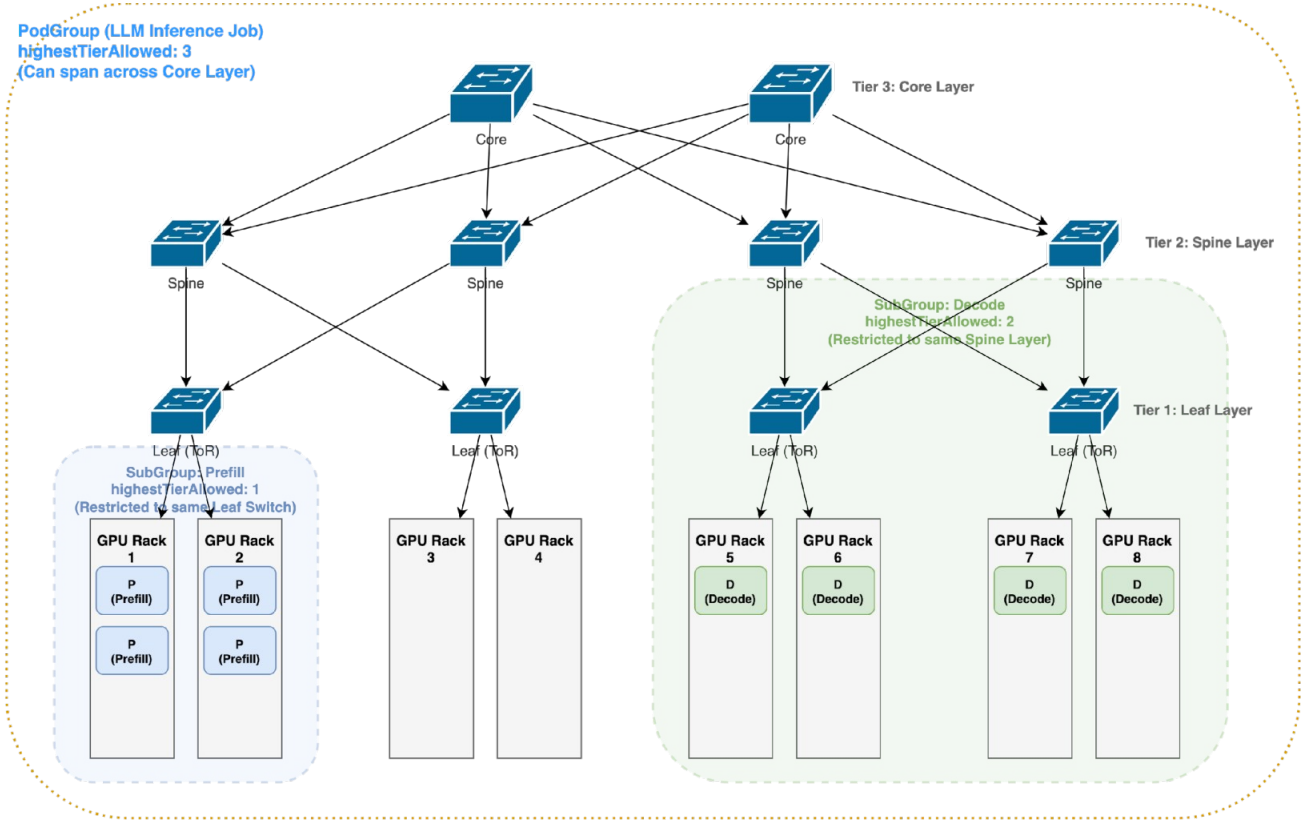
Leaf · HyperNode.spec

```
tier: 1
tierName: rack
members:
- type: Node
  selector:
    labelMatch:
      matchLabels:
        rack: rack-a
```

The same hierarchy is available through the standalone HyperNode component.

Place Groups at Different Topology Levels

An eight-Pod instance spans a larger domain; Prefill and Decode use different locality constraints.



Select each role's Pods in its policy; selectors are omitted here.

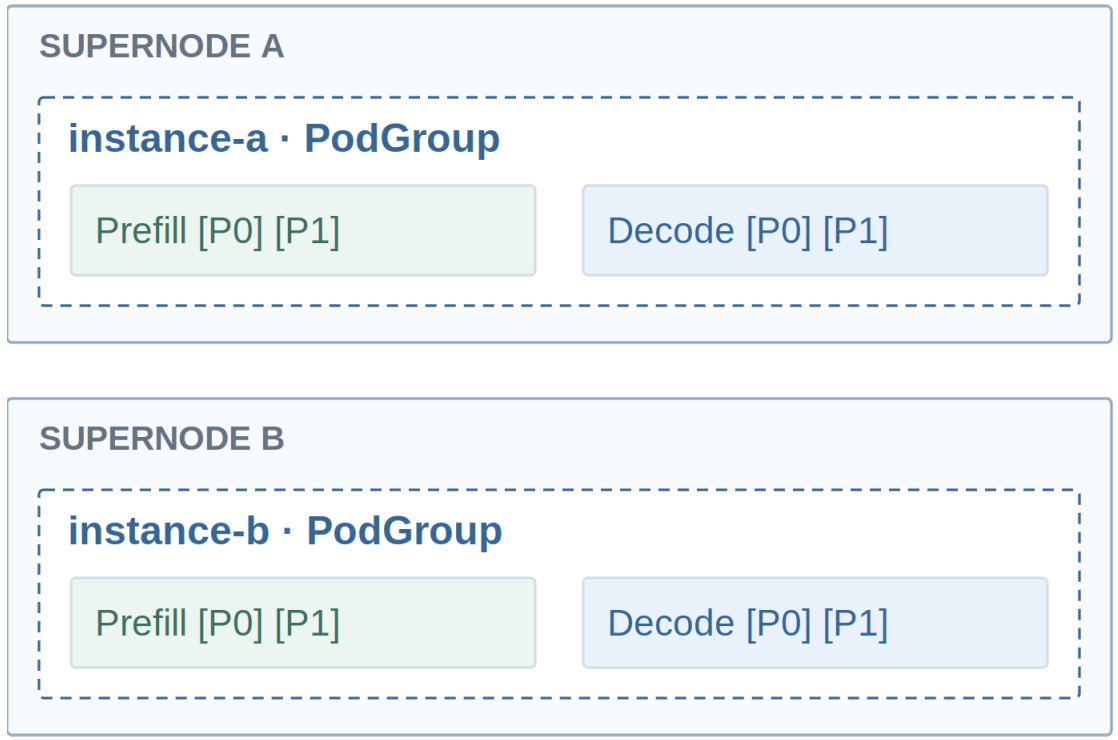
PodGroup.spec · key fields

```
minMember: 8
networkTopology:
  mode: hard
  highestTierAllowed: 3
subGroupPolicy:
- name: prefill
  subGroupSize: 4
  networkTopology:
    mode: hard
    highestTierAllowed: 1
- name: decode
  subGroupSize: 4
  networkTopology:
    mode: hard
    highestTierAllowed: 2
```

Keep Each Instance Together, Place Instances Apart

PodGroup anti-affinity separates service instances across HyperNode domains.

AVAILABLE · v1.16



PodGroup · same rule on each instance

```yaml
metadata:
  labels: {app: model-a}
spec:
  networkTopology:
    mode: hard
    highestTierName: supernode
  topologyAffinity:
    podGroupAntiAffinity:
      required:
      - podGroupSelector:
          matchLabels: {app: model-a}
        topologyTierName: supernode
```

Required rules wait for a legal domain. Preferred rules allow a feasible fallback.

03

Ecosystem

01 Unified scheduling

02 Multi-level gang & network topology

03 Ecosystem

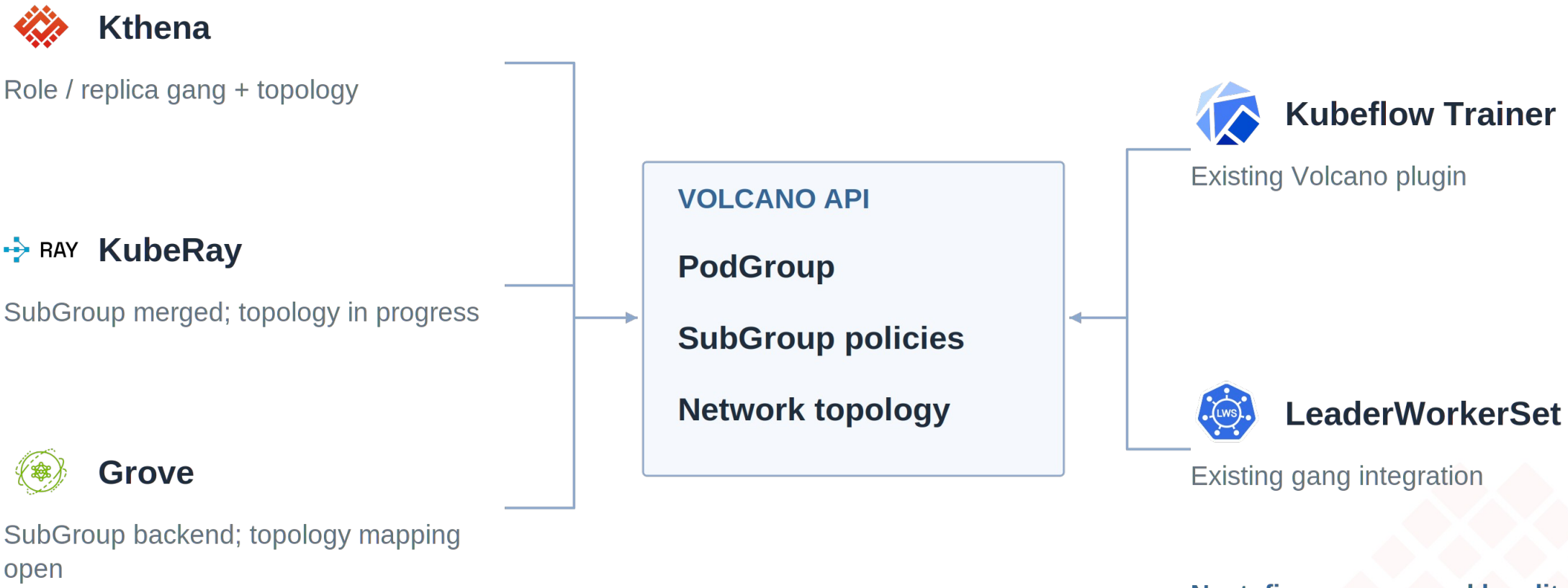
04 Benchmark framework & progress

Workload controllers carry scheduling intent.

How do roles and replicas reach the scheduler?

Controllers Bring Workload Structure to Scheduling

Integration depth differs: group support and topology support are separate steps.

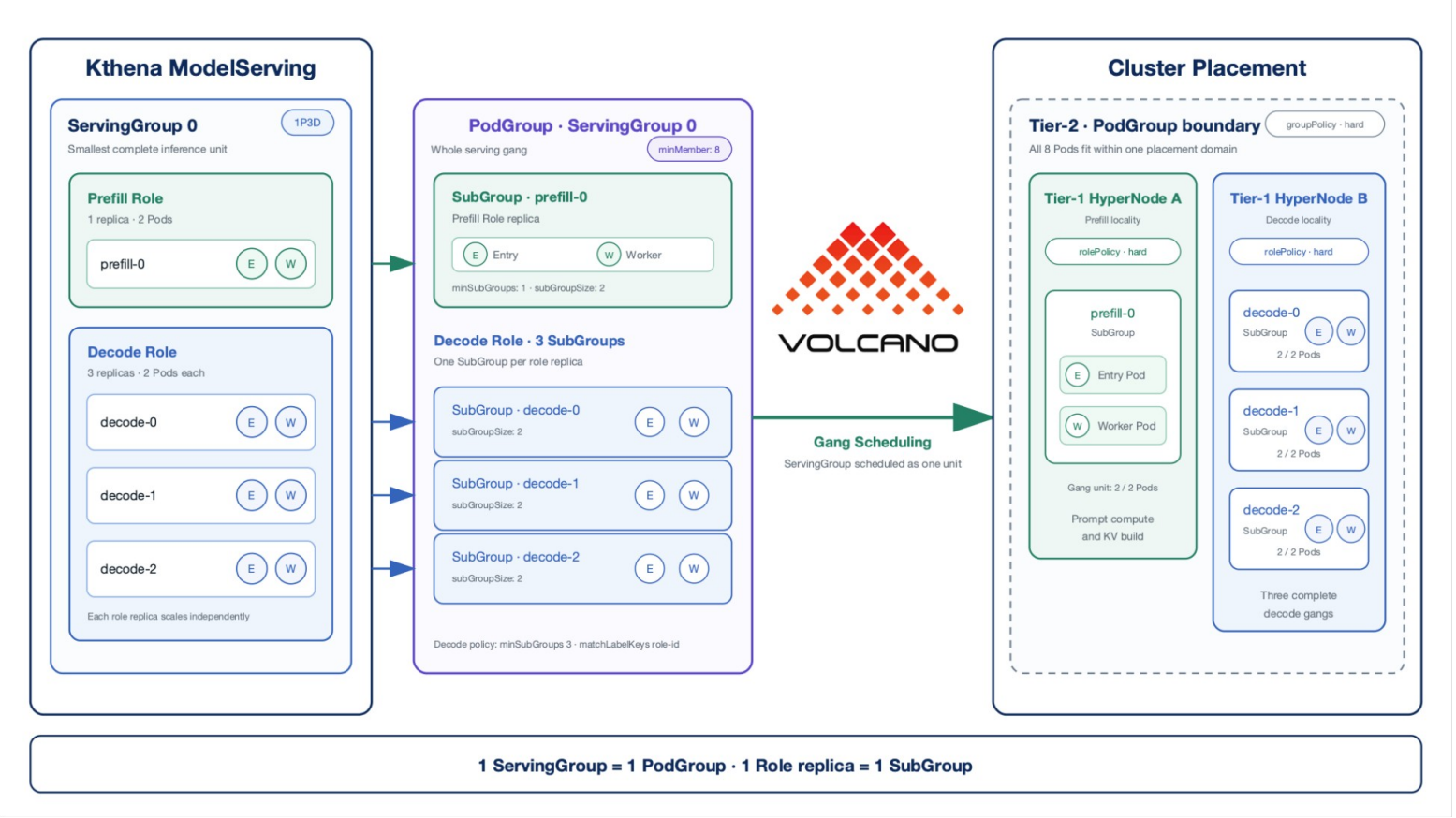


Next: finer groups and locality

Controllers preserve role and replica identity; Volcano applies scheduling policy.

Kthena: From Serving Roles to Scheduling Groups

One ServingGroup maps to one PodGroup; each role replica forms a SubGroup.



ModelServing.spec · excerpt

```
schedulerName: volcano
replicas: 1
template:
  gangPolicy:
    minRoleReplicas:
      prefill: 1
      decode: 3
  roles:
  - name: prefill
    replicas: 1
    workerReplicas: 1
  - name: decode
    replicas: 3
    workerReplicas: 1
```

Pod templates omitted.

KubeRay: A Replica Can Span Multiple Pods

The replica label separates complete groups inside one WorkerGroup.

RayCluster workerGroupSpecs · excerpt

Multi-host WorkerGroup

```
minReplicas: 2
numOfHosts: 2
```

replica-0 → SubGroup

host 0 · Pod | host 1 · Pod

replica-1 → SubGroup

host 0 · Pod | host 1 · Pod

Generated PodGroup.spec · excerpt

```
subGroupPolicy:
- name: worker
  labelSelector:
    matchLabels: {ray.io/group: worker}
  matchLabelKeys:
  - ray.io/worker-group-replica-name
  minSubGroups: 2
  subGroupSize: 2
```

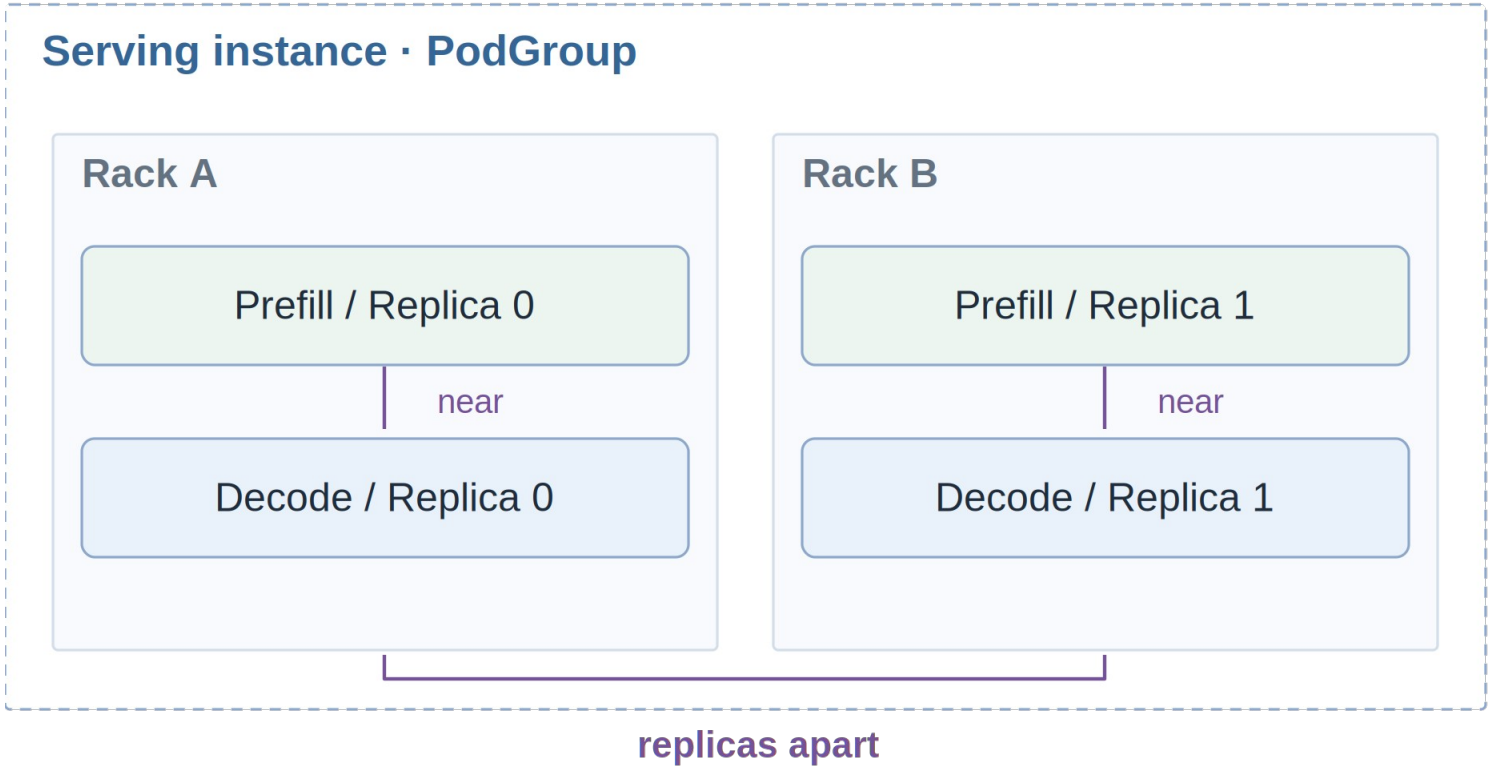
Gang: merged · Replica topology: in progress

The head uses a separate group.
One WorkerGroup can create multiple SubGroups.

Next: Relationships Between SubGroups

Extend the placement model as controllers expose richer workload structure.

PROPOSED · within one PodGroup



01 Affinity

Keep cooperating SubGroups in a shared domain.

02 Anti-affinity

Separate replicas across selected domains.

Controller mappings and API semantics are still being developed.

04

Benchmark framework & progress

Measure behavior across workload shapes.

What can we reproduce, compare, and improve?

01 Unified scheduling

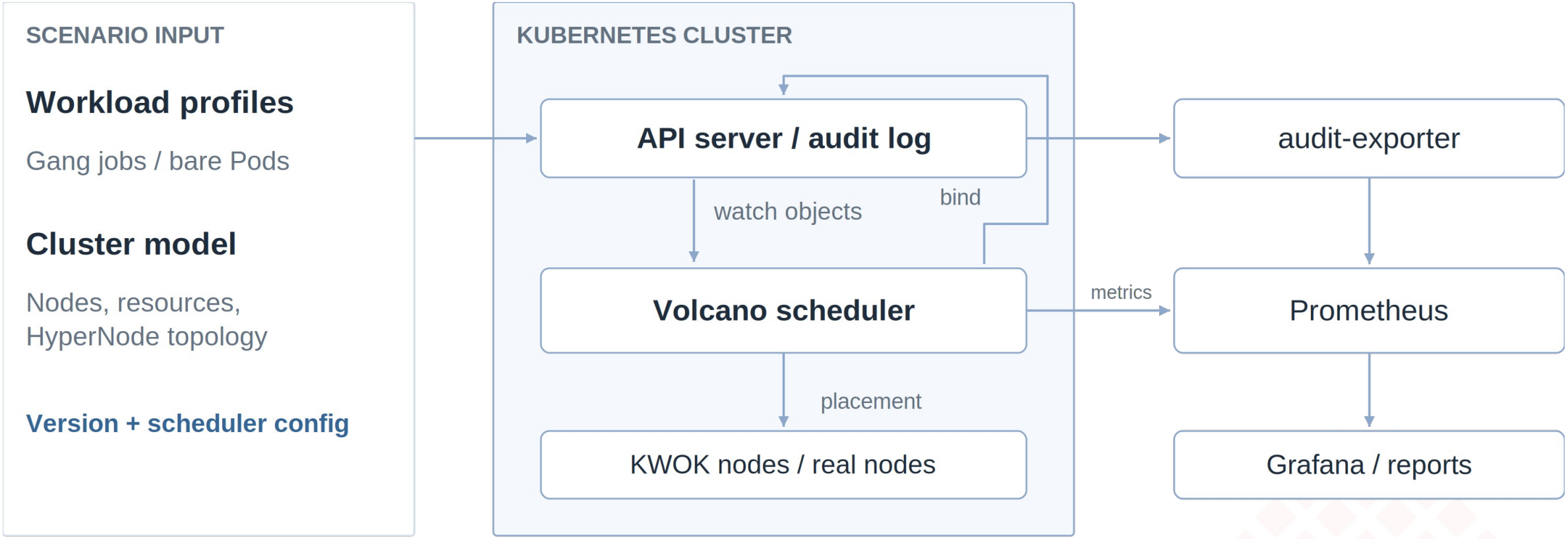
02 Multi-level gang & network topology

03 Ecosystem

04 Benchmark framework & progress

A Repeatable Scheduling Benchmark Framework

Generate scenarios, observe API events, and retain comparable measurements.



KWOK exercises scheduling behavior; real workloads are needed for GPU / network performance.

Benchmark Progress and Ongoing Validation

The harness is available; testing and optimization continue across workload scenarios.

benchmark/ · example workflow

```
# Build from local source
make setup

# A gang workload scenario
make test-gang-env \
  JOBS=10 REPLICAS=100 \
  MIN_AVAILABLE=100
```

AVAILABLE

Kind / existing clusters · Gang / Pod profiles
Topology setup · Audit / metrics · Reports

01 Vary the workload

Job size, total Pods, and placement constraints.

02 Compare consistently

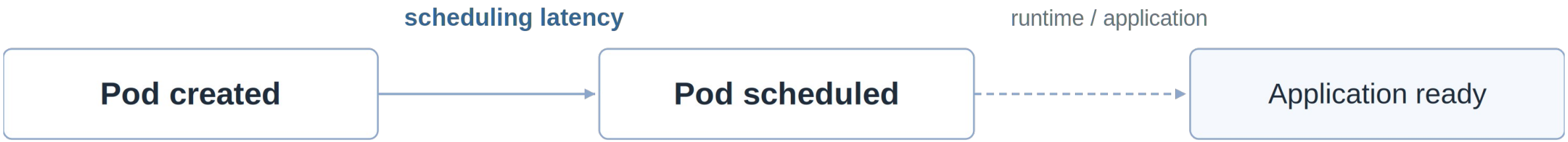
Pin versions, resources, and measurement windows.

03 Repeat and investigate

Re-run observations and trace the bottlenecks.

From Measurements to Reliable Comparisons

Keep scheduling latency distinct from application readiness.



Latency distribution

P50 / P90 / P99
Include the workload and scale.

Scheduling throughput

State the observation window.
Keep the raw events.

Comparison context

Version, configuration, resources and repeat runs.

Current focus: repeatable comparisons and continued validation.

Framework, Workloads, Ecosystem, Evidence

Four connected pieces of the scheduling story.

01 Unified scheduling

Coordinate node access across specialized scheduling paths.

02 Workload-aware placement

Preserve complete groups, locality, and instance separation.

03 Ecosystem integration

Bring controller roles and replica identity into scheduling.

04 Benchmark and improve

Use repeatable measurements to guide ongoing validation.

Questions & Discussion

Workload shapes, integration feedback, and reproducible observations are welcome.

What must your workload keep together—and what should it place apart?

github.com/volcano-sh/volcano

Roadmap and feedback · issue #6166